Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych

Projekt zespołowy – laboratorium 2

Implementacja bazy danych szeregów czasowych w języku Rust

Architektura systemu

Autorzy: Hubert Błonowski
Jan Zaborski

Promotor: mgr inż. Paweł Sobótka

Wersja dokumentu: 1.5
Data: 2 listopada 2026 r.

`https://github.com/janzaborski/rust-tsdb`

Streszczenie

Dokument przedstawia architekturę jednowęzłowej TSDB: moduły, model danych, współbieżny zapis, WAL, checkpoint, zapytania HTTP i integrację z Grafaną. Projekt łączy TSMap i bloki inspirowane Gorilla z WAL-em gałęzi `23-wal-improvements`. Jest specyfikacją docelową; parametry wydajnościowe są celami do weryfikacji. Pełną wersję sprzed skrócenia zachowano do porównania jako `lab2-Blonowski-Zaborski-pelna.pdf`.

Historia zmian

Wersja	Data	Autorzy	Zakres
1.0–1.3	24.10–02.11.2026	H. Błonowski, J. Zaborski	Projekt modułów, TSMap/TS, diagramy i kontrakty.
1.4	02.11.2026	H. Błonowski, J. Zaborski	Dostosowanie do WAL-a z 23-wal-improvements.
1.5	02.11.2026	H. Błonowski, J. Zaborski	Wersja do oceny; szczegóły implementacyjne w osobnym dokumencie.

Spis treści

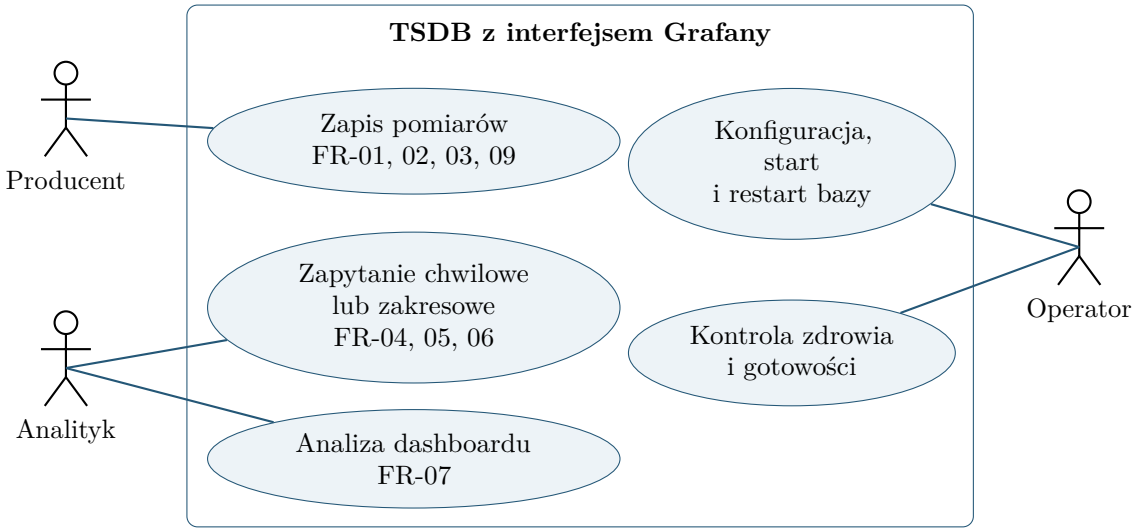
1 Cel, zakres i główne decyzje . . . 1
2 Architektura i odpowiedzialności modułów . . . 1
3 Model danych i współbieżność . . . 3
4 Ścieżka zapisu i architektura WAL . . . 4
5 Checkpoint, pliki i odzyskiwanie . . . 5
6 Zapytania i komunikacja HTTP . . . 6
7 Technologie, wdrożenie i interfejs użytkownika . . . 7
8 Weryfikacja, ryzyka i podział pracy . . . 9
Źródła . . . 9

1 Cel, zakres i główne decyzje

Celem jest własny silnik do przyjmowania pomiarów, przechowywania historii i wykonywania agregacji czasowych. Użytkownik analizuje dane w Grafanie. Zakres z lab1 [1] obejmuje HTTP, etykiety, pięć agregacji, out-of-order w otwartym buforze oraz demonstrację w Docker Compose.

System jest modularnym monolitem na jednym węźle. Jedna TSMap kataloguje szeregi TS, każdy z osobną blokadą; projekt nie wymaga shardów. Niekompresowane bloki aktywne i zamrożone umożliwiają zapis podczas flush. Poza zakresem pozostają klaster, replikacja, kompresja, retencja, kompaktowanie, alertowanie i własny frontend. Dane rosną do usunięcia środowiska; backup wymaga zatrzymania bazy i skopiowania katalogu.

WAL przejmujemy z `23-wal-improvements`, commit `e1354b9` [2]: bufor RAM, flusher, okresowy syncer, None/Written, lokalny LSN i cut. Rozwijamy pozostały silnik wokół tych mechanizmów. Wymagane korekty integracyjne opisuje sekcja 4.1.

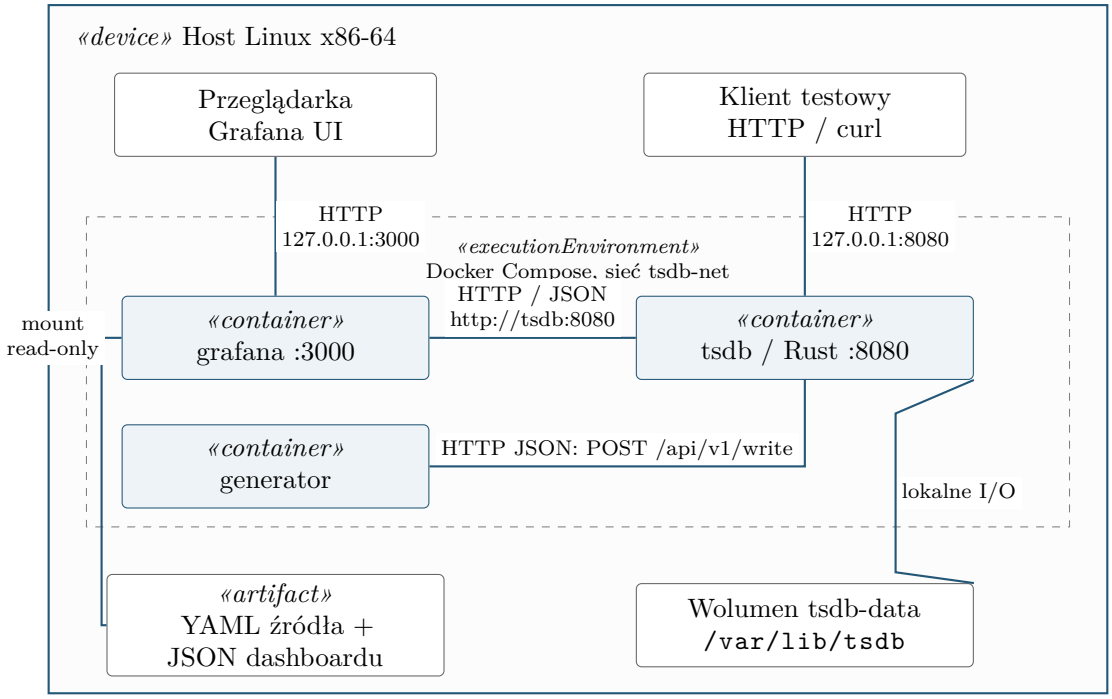


Rysunek 1: Diagram przypadków użycia UML. Flush (FR-08) jest automatycznym mechanizmem silnika, a nie osobnym działaniem użytkownika.

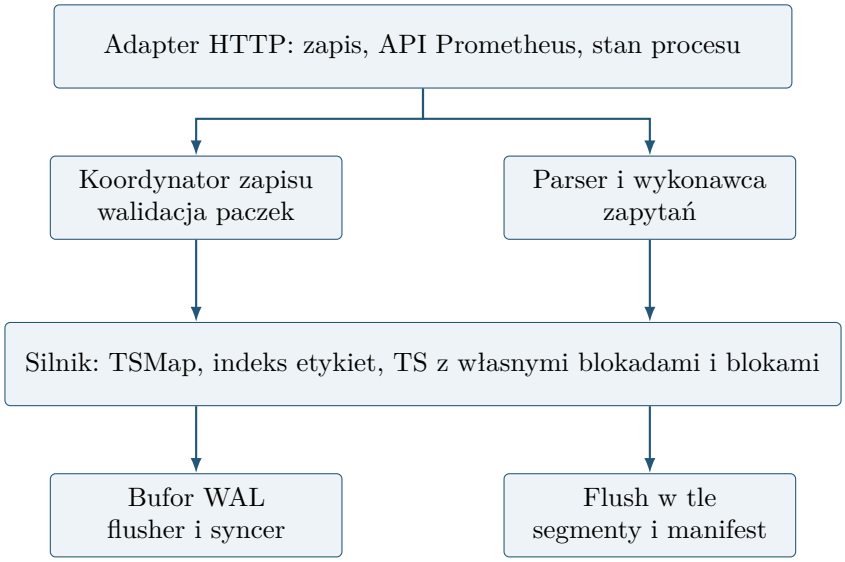
2 Architektura i odpowiedzialności modułów

Przeglądarka komunikuje się z Grafaną; jej serwer odpytuje TSDB. Generator przesyła pomiary bezpośrednio do bazy. Serwer Prometheus nie jest potrzebny. Silnik korzysta z lokalnego systemu plików.

Adapter HTTP zna protokół, parser zna składnię, a silnik zna dane i pliki. Dzięki temu magazyn można rozwijać/testować bez HTTP, a wykonawcę zapytań z pamięciowym interfejsem odczytu.



Rysunek 2: Wdrożenie na jednym hoście. Kontenery używają DNS tsdb, a porty hosta są publikowane wyłącznie na loopback.

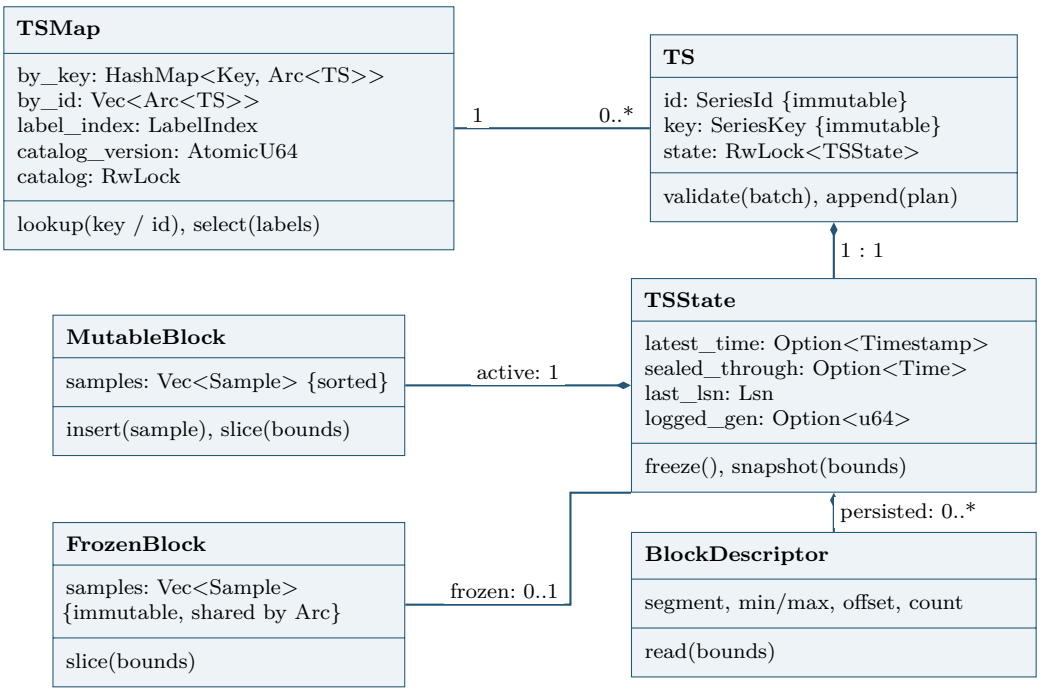


Rysunek 3: Zależności logiczne. Konfiguracja i diagnostyka są współdzielonymi usługami pomocniczymi.

Moduł	Odpowiedzialność i kontrakt
HTTP i walidacja	Dekodowanie, kanonizacja, limity i błędy; wywołanie zapisu lub zapytania.
Koordynator zapisu	Walidacja całej paczki, rezerwacja zasobów, kolejność blokad i wywołanie WalWriter.
TSMap, TS i indeksy	Klucz ↔ TS, dobór po etykietach, aktywny blok oraz uchwyty frozen/segmentów.
WAL	Append do RAM, postęp Written, zapis/sync w tle, cut i replay.
Magazyn segmentów	Niezmienne pliki, manifest, checkpoint i recovery.
Parser i wykonawca	Tekst → AST → ReadView → wynik; limity czasu, pamięci i pracy.

3 Model danych i współbieżność

Szereg identyfikuje nazwa metryki i kanonicznie posortowane etykiety. Wewnętrzny LabelSet zawiera także `__name__`. Punkt to czas Unix w milisekundach (u64) i skończona wartość f64; para zajmuje 16 B. SeriesId jest lokalny dla head/WAL i może zmienić się po restarcie. Segmenty i manifest identyfikują szeregi przez klucze.



Rysunek 4: Model head. Romb: własność; linia: asocjacja. Key = SeriesKey, Time = Timestamp; próbka zajmuje 16 B.

Inspiracją z Gorilla [3] jest oddzielenie katalogu, synchronizacji TS i bloków. Active to posortowany wektor: rosnące czasy dopisuje na końcu, out-of-order wymaga wstawienia. Freeze przenosi wektor do współdzielonego Arc bez pełnej kopii; nowy active przyjmuje pomiary. ReadView kopiuje potrzebne active i współdzieli niezmienne bloki.

3.1 Reguły spójności

Zapis jest atomowy dla całej paczki, także obejmującej kilka TS. Dla nowych kluczy najpierw pobiera się mutex rejestracji. Każda paczka pobiera następnie współdzieloną barierę generacji i

wyłączne blokady TS rosnąco po ID. Rezerwacja zasobów poprzedza append. Kolejność: WAL append, publikacja całej paczki, zwolnienie blokad i dopiero ewentualne oczekiwanie Written. Blokady katalogu nie obejmują kodowania ani I/O.

Odczyt wybiera uchwyty i wersję katalogu, następnie pod współdzielonymi blokadami wybranych TS tworzy jeden ReadView dla całego zapytania. Zmiana wersji katalogu wymusza ponowienie. Odczyt nie widzi części paczki; może widzieć paczkę klienta nadal czekającego na Written. Widoczność i trwałość są odrębne.

3.2 Duplikaty i opóźnione punkty

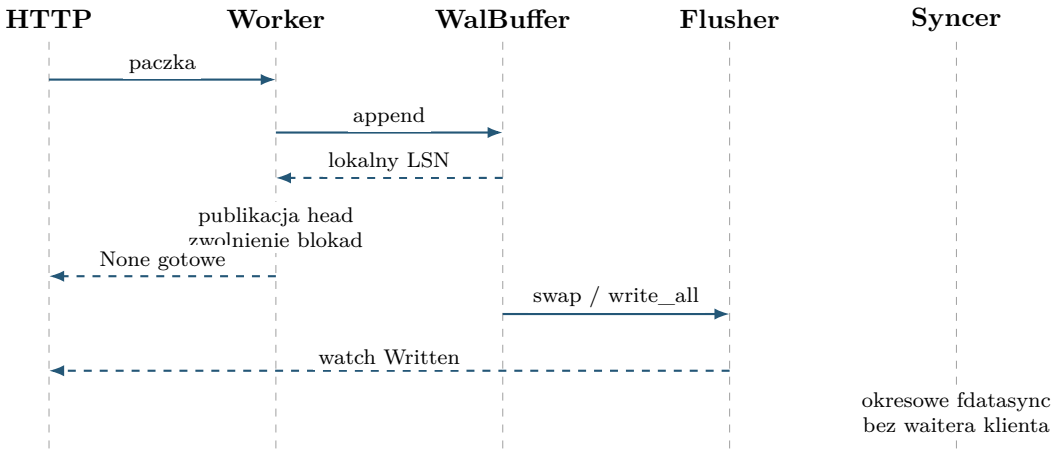
Identyczny punkt w active jest duplikatem bez zmian; inna wartość dla tego samego czasu odrzuca całą paczkę jako konflikt. Nie modyfikuje się zamkniętych zakresów, także dla identycznego duplikatu. Dla nowego punktu obowiązuje

$$t > S_s \quad \wedge \quad t \geq H_s - W,$$

gdzie H_s to największy przyjęty czas, S_s granica zamknięcia, a W okno opóźnienia (domyślnie 5 min). Całą paczkę porównuje się ze stanem sprzed jej przyjęcia. Warunki nieustalonego H_s/S_s pomija się. Próbka może wyprzedzać zegar serwera najwyżej o 60 s.

4 Ścieżka zapisu i architektura WAL

Żądanie najpierw rezerwuje przyjęcie i pamięć, waliduje całą paczkę, a potem wywołuje WalWriter.append. Encoder tworzy jedną ramkę w thread-local scratch: definicje szeregów, następnie grupy próbek. Mutex bufora chroni tylko kopię ramki i przyrost LSN. Błędy walidacji/przeciążenia występują przed append; po nim nie ma zwykłego rollbacku, również przy timeout lub utracie połączenia.

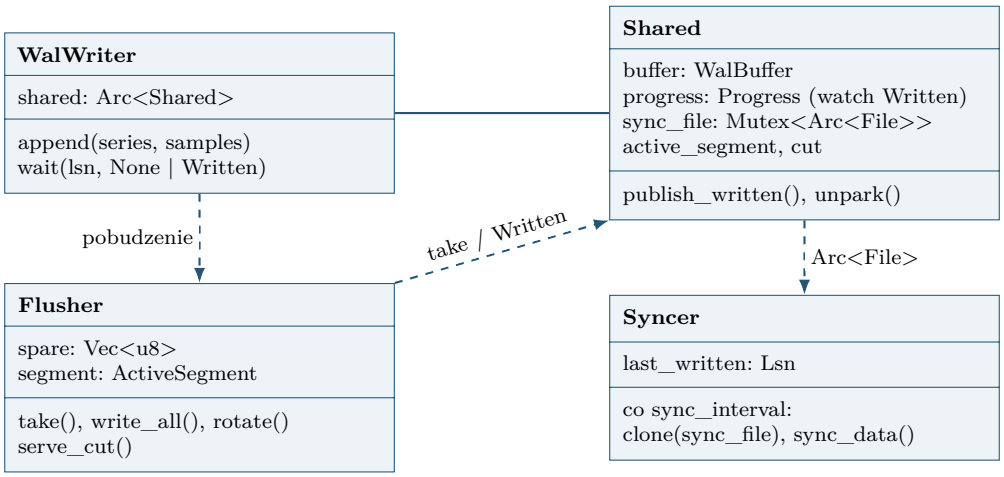


Rysunek 5: Ścieżka zapisu zgodna z WalWriter. None kończy po head append, Written po write_all; okresowy syncer nie potwierdza trybu per żądanie.

Tryb	Znaczenie odpowiedzi po publikacji head
None (domyślny)	Bez czekania na dysk; awaria procesu może utracić paczkę.
Written	Pełna ramka zapisana do OS/page cache; przetrwa awarię samego procesu przy sprawnym hoście/nośniku.

Nie ma publicznego Synced. Okresowy sync nie zapewnia potwierdzenia per żądanie ani gwarantowanego okna straty przy awarii hosta. LSN to liczba bajtów przyjętych w jednym Wal::open; po

restarcie zaczyna od 0 i nie jest trwałym kursorem recovery.



Rysunek 6: Typy i odpowiedzialności WAL-a z brancha. Syncer nie publikuje postępu do klienta; cut obsługuje flusher.

Flusher podmienia Vec na spare i poza mutexem wykonuje write_all, publikując Written przez watch. Domyślne parametry brancha: flush 100 ms lub 4 MiB, group commit do 200 μs dla porcji poniżej 256 KiB, sync co 1 s. Próg pliku 128 MiB jest miękki: rotacja następuje przed następną porcją, synchronizuje stary plik i tworzy nowy. Rekordy nie są dzielone między pliki.

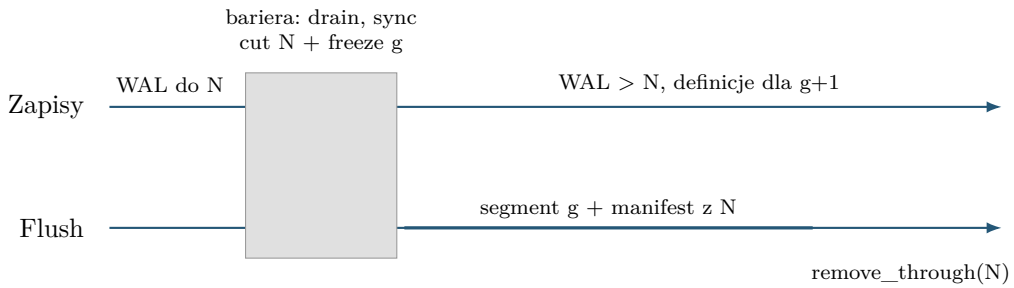
4.1 Uzupełnienia wymagane przy integracji

Zachowujemy ideę i kodek brancha. Do wdrożenia pozostają: filtrowanie replay po checkpointcie z manifestu; odcinanie tylko niepełnego końca ostatniego pliku; twardy budżet bufora/scratch; błąd przy zamkniętym watch zamiast sukcesu; bariera cut i logged_gen; blokada gotowości przy osieroconych próbkach lub błędzie sinka. Pełne kontrakty tych korekt opisano w wersji sprzed skrócenia.

5 Checkpoint, pliki i odzyskiwanie

Flush zleca timer po 30 min pobytu najstarszej próbki generacji w RAM albo przekroczenie 80% budżetu head. Działa jeden flush, bez kolejki zamrożonych generacji. Przy przeciążeniu zapis krótko czeka bez blokad lub dostaje 503 przed append.

1. Wyłączna bariera czeka na rozpoczęte WAL append–head append. Bez blokad TS wykonuje Wal.cut: drain, sync i rotacja; wynik to ID zamkniętego pliku N. Na czas cut nowe zapisy czekają na I/O.
2. Freeze przenosi active do frozen, podnosi S_s i otwiera generację $g + 1$. Po zwolnieniu bariery nowe zapisy idą do WAL $> N$. Pierwszy zapis każdego TS ponownie loguje definicję (logged_gen), nawet gdy TS istniał przed cut.
3. W tle: segment danych i jego sync, potem atomowo opublikowany i zsynchronizowany manifest z checkpointem N.
4. Zamiana frozen na deskryptory, potem remove_through(N). Pliki $> N$ i aktywny WAL pozostają; starsze ReadView zachowują Arc.



Rysunek 7: Granica checkpointu pokrywa całe pliki WAL. Pauza obejmuje I/O cut, ale zapis segmentu i manifestu jest poza barierą.

Artefakt	Format i rola
WAL	Pliki wal-00000001.log bez nagłówka; ramka `len|crc32|kind|payload`, nagłówek 9 B, CRC-32/IEEE. Len oznacza payload; kind Write=0.
Segment danych	Docelowy binarny format v3: słownik LabelSet, bloki próbek, indeks zakresów i CRC; próbki odczytywane wycinkami.
MANIFEST	Docelowa koperta v4 z JSON/CRC: segmenty, klucze, granice zamknięcia i wal_checkpoint=N; bez trwałego LSN/head ID.

Recovery uzyskuje LOCK katalogu, sprawdza manifest/segmenty i odbudowuje Index po kluczach. Pomija WAL $\leq N$ nawet jeśli zostały po awarii; odtwarza ciąg plików $> N$ od offsetu 0. Resolver mapuje ID na wynik ReplaySink.series(LabelSet), scala ten sam klucz i buforuje próbki wcześniejsze od definicji. Brak definicji na końcu blokuje gotowość. Obcina się tylko niepełny ogon ostatniego pliku; pełne błędne CRC zatrzymuje start. Po replay otwierany jest nowy WAL z LSN=0.

Błąd I/O WAL kończy proces. Błąd segmentu zatrzymuje przyjęcia, zachowując head, WAL i odczyt do diagnostyki. SIGTERM zamyka przyjęcia, kończy rozpoczęty cut i zadania, następnie Wal.close drenuje/synchronizuje bufor i dołącza wątki. Pełny flush head nie jest konieczny.

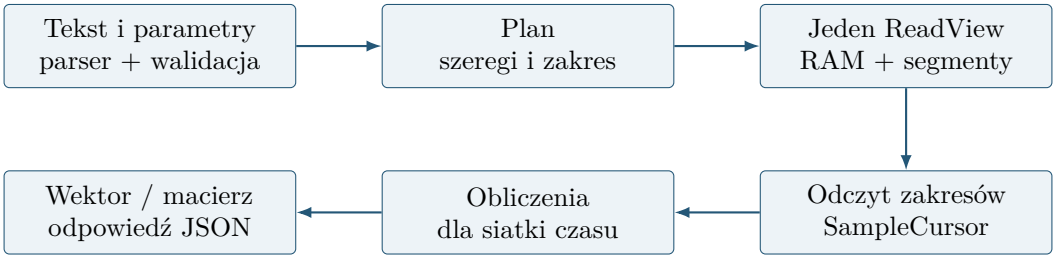
6 Zapytania i komunikacja HTTP

Obsługiwany podzbiór PromQL [4] obejmuje nazwany selektor z dokładnym dopasowaniem etykiet, pięć funkcji czasowych i prosty skalar (np. 1+1 do próby połączenia). Bez regexów, zagnieżdżeń i agregacji między szeregami.

```
temperature_celsius{service="api"}
avg_over_time(temperature_celsius{service="api"}[5m])
```

Pozostałe funkcje: `min_over_time`, `max_over_time`, `sum_over_time`, `count_over_time`. Selektor wybiera ostatnią próbkę z $[T - D, T]$ (lookback $D = 5$ min). Funkcja działa osobno dla TS na $(T - R, T]$; puste okno pomija TS, również dla count. Range powtarza obliczenie dla $T_k = start + k \cdot step \leq end$, używając jednego ReadView. Indeks etykiet wybiera TS, indeksy czasowe ograniczają odczyt, a agregacje korzystają z przesuwanego okna.

HTTP/1.1, UTF-8 i JSON; zapytania także przez formularz POST. Wartości wyników są napisami, czasy wyników sekundami Unix. Zapis przyjmuje milisekundy i skończone f64.



Rysunek 8: Przepływ zapytania. Każdy etap może zakończyć się błędem lub przekroczeniem budżetu.

Interfejs	Główny kontrakt
POST `/api/v1/write`	Jeden pomiar albo `points`; opcjonalne durability=none|written, cała paczka atomowa.
GET/POST `/api/v1/query`	query, opcjonalny time; wynik chwilowy.
GET/POST `/api/v1/query_range`	query, start, end, dodatni step; macierz wyników na siatce czasu.
GET `/api/v1/labels`, `/api/v1/label/<name>/values`	Nazwy/wartości etykiet z całego katalogu.
GET `/api/v1/status/buildinfo`, `/healthz`, `/readyz`	Wersja TSDB, zdrowie i gotowość po recovery.

```
POST /api/v1/write?durability=written
{"metric":"temperature_celsius","labels":{"service":"api"},
 "timestamp":1792843200000,"value":24.5}
```

Sukces zapisu: inserted, duplicates, lokalny lsn i durability. Zapytania stosują kopertę Prometheusa i vector/matrix/scalar. Błędy: 400 walidacja/składnia, 409 konflikt, 413 rozmiar, 422 reguły wykonania, 503 przeciążenie/gotowość/timeout. Timeout nie wycofuje zapisu.

7 Technologie, wdrożenie i interfejs użytkownika

Wybór	Uzasadnienie i rozważana alternatywa
Rust 2024, Arc/RwLock/Mutex	Kontrola własności i pamięci; zamiast Go/C++. Własne limity nadal konieczne.
Axum 0.8, Tokio 1	Async HTTP i routing; zamiast Actix/bezpośrednio Hyper. Plikowe I/O poza wątkami sieci.
Serde 1, time 0.3, Clap 4	DTO/JSON, RFC 3339 i CLI; zamiast ręcznych parserów i dodatkowego formatu konfiguracji.
std::fs, libc 0.2, crc32fast 1	Własny silnik jest celem projektu; zamiast RocksDB/SQLite, kosztem odpowiedzialności za recovery.
thiserror 2, tracing 0.1	Typowane błędy i logi; zamiast własnego stosu.
Grafana OSS 13.2.0, Compose v2	Gotowe wykresy i powtarzalne środowisko; zamiast własnego GUI/Kubernetes.
cargo test, Tower 0.5, reqwest 0.12	Testy modułów/HTTP i generator w Rust, bez osobnego frameworka obciążeniowego.

Wersje utrwala Cargo.lock, obraz Grafany jest przypięty. Platforma docelowa: Linux x86-64 i lokalny system plików z rename oraz sync plików/katalogów [6]. LOCK dopuszcza jeden proces na

katalog. Usługi tsdb/grafana/generator korzystają z sieci tsdb-net i wolumenu tsdb-data. Grafana odpytuje `http://tsdb:8080`; przeglądarka korzysta z portu 3000. API testowe 8080 i Grafana są publikowane na loopback. Baza działa bez root; API ufa lokalnej sieci, bez auth/TLS. Grafana ma logowanie, a sekret jest poza repozytorium.

7.1 Konfiguracja i budżety

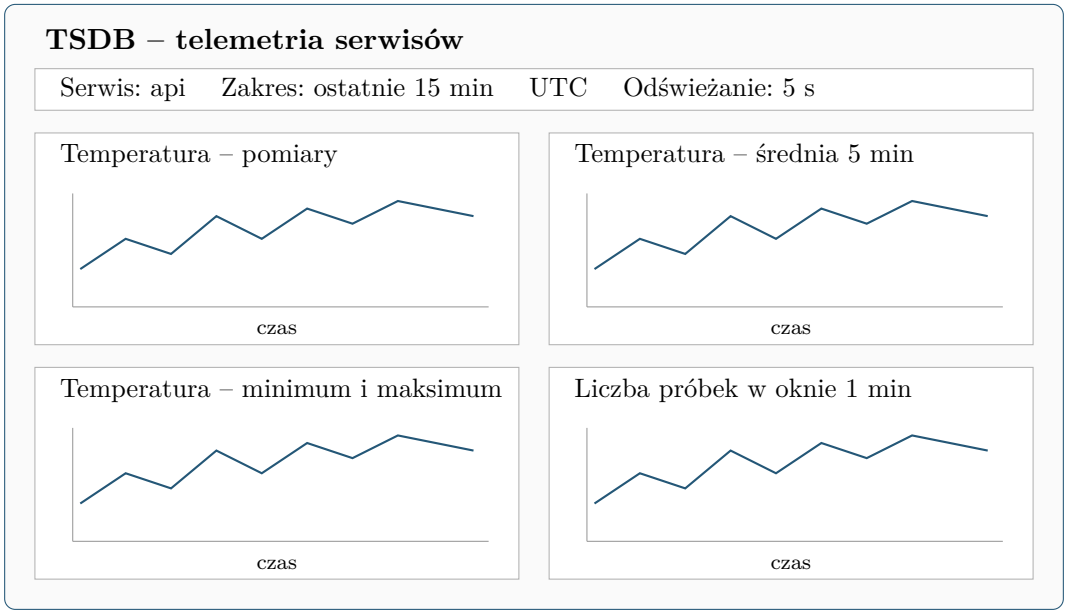
Konfiguracja CLI jest wczytywana przy starcie; Compose przekazuje ją w command. Kluczowe wartości:

Obszar	Wartości domyślne / ograniczenia
Head i katalog	512 MiB active/frozen/Arc, metadata 64 MiB, 10 000 TS; buforowanie 30 min, out-of-order 5 min.
WAL i przyjęcie	32 MiB active+spare, 64 MiB ciał/paczek/scratch; 8 przyjętych zapisów, 4 workers.
Paczka	Do 10 000 punktów, ciało 4 MiB, ramka HTTP do 8 MiB.
Zapytanie	2 równoległe; po 64 MiB, 30 s, 1000 TS, 2 mln wejściowych próbek i 1 mln punktów wyniku.

Pamięć frozen jest liczona do ostatniego Arc, także po publikacji segmentu; realokacje i zapas pojemności podlegają budżetom. Limity head nie są limitem RSS całego procesu. Logi obejmują request ID, wynik, czas, LSN/tryb, backlog, generację/checkpoint, cut/flush oraz raport recovery.

7.2 Dashboard i demonstracja

Grafana jest jedynym GUI. Provisioning [5] dostarcza źródło Prometheus (proxy, POST, UID tsdb) i dashboard. Wybór jednego serwisu api/worker, UTC, zakres ostatnich 15 min, odświeżanie 5 s; instancje są osobnymi liniami.



Rysunek 9: Makieta układu dashboardu. Linie są ilustracją, nie wynikami eksperymentu.

Cztery panele: pomiary, avg_over_time z 5 min, min/max z 5 min, count_over_time z 1 min. Zmienna jest typu Custom bez All/multiselect, a zapytania w trybie Code, aby nie wymagać regexów i pełnego PromQL. Brak danych jest przerwą/No data, błąd API pozostaje błędem panelu.

Demo: uruchomienie generatorów po readyz, zmiana serwisu/czasu, odczyt podczas flush oraz restart z zachowaniem wolumenu. Profil pokazowy skraca bufor do 1 min i opóźnienie do 30 s; testy zasobów zachowują profil lab1.

8 Weryfikacja, ryzyka i podział pracy

Testy obejmą atomowość paczek i kanonizację (FR-01/02), Written/recovery (FR-03), semantykę zapytań/funkcji (FR-04–06), Grafanę (FR-07), flush i out-of-order (FR-08/09). Wymuszone przeploty sprawdzą blokady, spójny ReadView, logged_gen, cut, snapshot katalogu i usuwanie WAL. Awarie w Written (co najmniej 20 prób z dziennikiem klienta) obejmą zapis ramki, cut, manifest i usuwanie plików.

Cel z lab1	Profil pomiaru
p95: zapis 100 ms, paczka 500 ms; odczyt 200 ms, agregacja RAM 500 ms	HTTP z kolejką i Written; paczka 1000 punktów, odczyt 1000 wyników.
Co najmniej 10 000 punktów/s	1000 TS, paczki po 1000, 4 klientów, Written, godzina z flush.
RSS do 1 GiB	10 000 punktów/s, 30 min buforowania, minimum 65 min, także podczas flush i z dwoma zapytaniami.
Recovery do 10 s	Milion potwierdzonych punktów, osobno WAL/segmenty oraz zimny/ciepły cache.

Referencja: Linux, co najmniej 4 rdzenie, 8 GiB RAM, lokalny SSD, build release. Raport podaje sprzęt, konfigurację i commit; None mierzy się oddzielnie od trwałej przepustowości. Główne ryzyka to pauza I/O cut, RAM podczas flush/odczytu, liniowy koszt out-of-order, wzrost plików bez retencji i zgodność Grafany. Ocenia się p95, RSS, backlog, czasy cut/flush i liczbę odrzuceń; cele wymagają pomiarów.

Jan Zaborski odpowiada za WAL, segmenty, checkpoint i recovery. Hubert Błonowski za parser, zapytania, HTTP i Grafanę. Model danych, kontrakty, testy integracyjne i pomiary są wspólne. Wersja sprzed skrócenia zachowuje scenariusze T-01–20, profile P-01–06, formaty bajtowe i komplet ustawień.

Literatura

[1] H. Błonowski, J. Zaborski, *Specyfikacja wymagań i zakres pracy*, lab1, wersja 1.1, 21.10.2026.

[2] Repozytorium projektu, `23-wal-improvements`, commit `e1354b9`, `src/storage/wal`, odczyt 02.11.2026. `https://github.com/janzaborski/rust-tsdb/tree/e1354b9/src/storage/wal`.

[3] T. Pelkonen i in., *Gorilla: A Fast, Scalable, In-Memory Time Series Database*, PVLDB 8(12), 2015, sekcje 4.2–4.3. `https://www.vldb.org/pvldb/vol8/p1816-teller.pdf`.

[4] Prometheus, *HTTP API*. `https://prometheus.io/docs/prometheus/latest/querying/api/`.

[5] Grafana Labs, *Provision Grafana*. `https://grafana.com/docs/grafana/latest/administration/provisioning/`.

[6] Linux man-pages, *fsync(2), fdatasync(2)*. `https://man7.org/linux/man-pages/man2/fsync.2.html`.